Computation for Analytics

DIANE WOODBRIDGE, PH.D

Contents

Modules, Packages and Libraries

Importing Libraries/Packages/Module

Standard Libraries/Packages/Modules

- Mathematical Modules
- Data Persistence
- File Formats
- Generic Operating System Services

Contents

Modules, Packages and Libraries

Importing Libraries/Packages/Module

Standard Libraries/Packages/Modules

- Mathematical Modules
- Data Persistence
- File Formats
- Generic Operating System Services

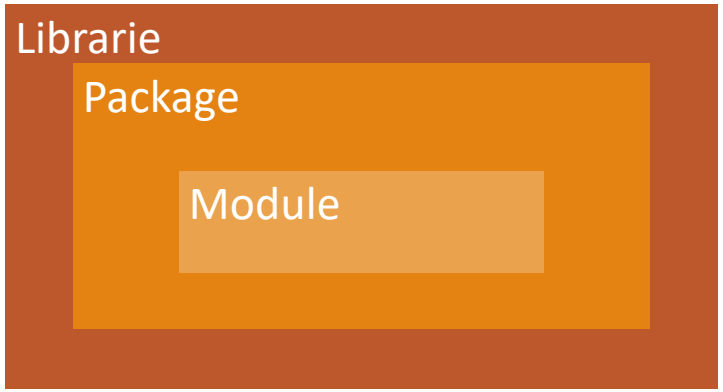
Modules, Packages and Libraries

Module

- A simple Python file (.py)

Packages

- A simple directory having collections of modules.
- It includes __init__.py as a constructor.

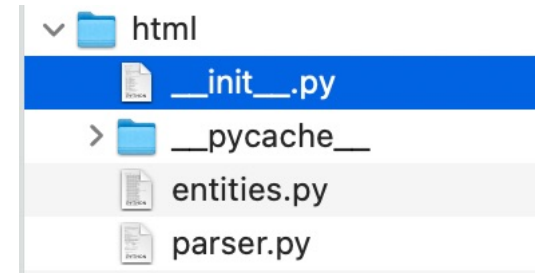


Libraries

- A collection of related functionality of codes / Collection of packages
- You can install it by `$conda install library_name` and import it using `import library_name`

__init__.py

In a package/library, __init__.py is considered to be a starting points which initialize the package/library.



Python's html library

__init__.py<html>

__init__.py

```
import re as _re
from html.entities import html5 as _html5


__all__ = ['escape', 'unescape']


def escape(s, quote=True):
    """
    Replace special characters "&", "<" and ">" to HTML-safe sequences.
    If the optional flag quote is true (the default), the quotation mark
    characters, both double quote (") and single quote (') characters are also
    translated.
    """
    s = s.replace("&", "&amp;") # Must be done first!
    s = s.replace("<", "&lt;")
        replace(">", "&gt;")
    te:
        s = s.replace('"', "&quot;")
```

https://docs.python.org/3/tutorial/modules.html

Standard Libraries/Packages/Modules

Python comes with extensive **built-in** libraries, packages and modules

- Mathematical Modules
- Data Persistence
- File Formats
- Generic Operating System Services
- Internet Protocols and Support
- Structured Markup Processing Tools
- And so many more…

https://docs.python.org/3/library/

Useful Data Science Packages

Those are not standard, which means that you need to install.

- NumPy : High-level mathematical functions
- SciPy : Scientific computing and technical computing
- Pandas : Data manipulation and analysis
- SciKit-Learn : Machine learning library
- Keras : Artificial neural networks
- TensorFlow : Machine learning and artificial intelligence
- PyTorch : Neural networks
- Matplotlib : Plotting library
- Plotly : Plotting library
- Scrapy : Web-crawling framework
- BeautifulSoup : Parsing HTML and XML



Contents

Modules, Packages and Libraries

Importing Libraries/Packages/Module

Standard Libraries/Packages/Modules

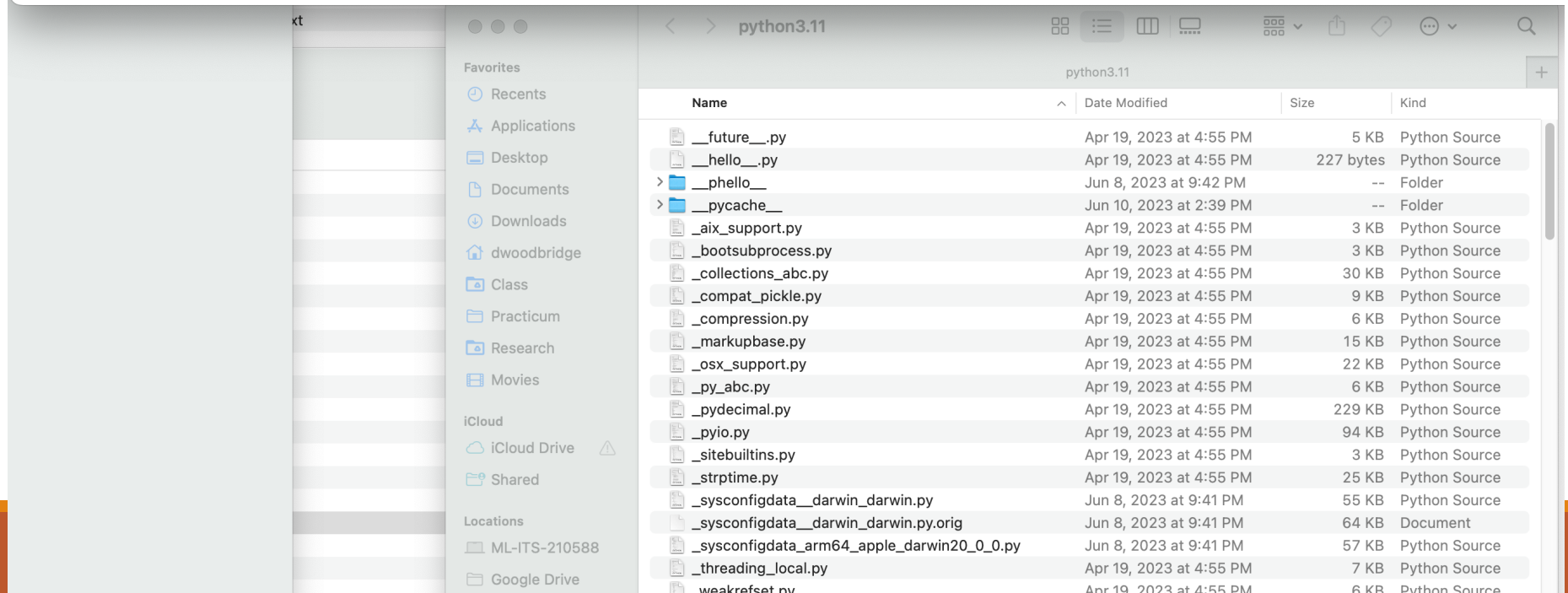
- Mathematical Modules
- Data Persistence
- File Formats
- Generic Operating System Services

Importing Libraries/Packages/Module

Where are they located?

- Under `your_environment/lib/python3.xx`

```
dwoodbridge@ML-ITS-210588 msds501_computation_2023 % which python
/opt/homebrew/anaconda3/bin/python
dwoodbridge@ML-ITS-210588 msds501_computation_2023 % open /opt/homebrew/anaconda3/lib/python3.11
dwoodbridge@ML-ITS-210588 msds501_computation_2023 %
```



Importing Libraries/Packages/Module

Many of the standard libraries on your disk can't be automatically loaded into memory

- Need to **explicitly** `import` at the beginning of your code.

- Syntax

```
import <module_name>
from <module_name> import <name> [as <alt_name>]
```

- Helpful Tips
  - dir() function
    - Returns included functions, modules, etc.
  - `help()` function
    - Display the documentation of modules, packages, functions, classes, keywords, etc.
    - Useful for understanding usage, input/output and examples.
  - Jupyter Lab/VSCode's auto-complete
    - If you hit tab after your module name, it shows available options.




```
import math
```

```
math.
```

```
f acos      function
f acosh     function
f asin      function
f asinh     function
```

Example 1

For the html package, see available functions and modules.

In the html package, see the parser function's documentation.

Contents

Modules, Packages and Libraries

Importing Libraries/Packages/Module

Standard Libraries/Packages/Modules

- Mathematical Modules
- Data Persistence
- File Formats
- Generic Operating System Services

Standard Libraries/Packages/Modules

Python comes with extensive built-in libraries, packages and modules

- Mathematical Modules
- Data Persistence
- File Formats
- Generic Operating System Services
- Internet Protocols and Support
- Structured Markup Processing Tools
- And so many more…

https://docs.python.org/3/library/

Python Libraries/Packages/Modules

There are so many and still expanding.

- I will cover a few that are used often.
  - You will often have to visit the documentation.
  - Also, autocomplete, dir() and help() function will be useful to see details.



Python Libraries/Packages/Modules

You can always implement things from scratch. 😔 (Ex. sum, count)

- BUT knowing available packages and modules saves a ton of time, as it provides high-level functions which you can simply pass input parameters.
- It is impossible to cover all the packages/libraries in a class. I can show what are available and you need to think about and figure out what to use while designing low-level code.



Contents

Modules, Packages and Libraries

Importing Libraries/Packages/Module

Standard Libraries/Packages/Modules

- **Mathematical Modules**
- Data Persistence
- File Formats
- Generic Operating System Services

Standard Libraries/Packages/Modules

Python comes with extensive built-in libraries, packages and modules

- Mathematical Modules
  - numbers — Numeric abstract base classes
  - **math** — Mathematical functions
  - cmath — Mathematical functions for complex numbers
  - decimal — Decimal fixed point and floating point arithmetic
  - fractions — Rational numbers
  - **random** — Generate pseudo-random numbers
  - statistics — Mathematical statistics functions

https://docs.python.org/3/library/math.html
https://docs.python.org/3/library/random.html

Standard Libraries/Packages/Modules

Python comes with extensive built-in libraries, packages and modules

- Mathematical Modules
  - **math** — Mathematical functions

Name	Explanation	Example

math.ceil(x)/floor(*x*)	Return the ceiling/floor of x.	`x = math.pi`
		`math.ceil(x)/floor(x)`
		`# 4, 3`

math.isclose(a, b, rel_tol=1e-09, abs_tol=0)	Return True if the values a and b are close to each other. abs(a-b) <= max(rel_tol * max(abs(a), abs(b)), abs_tol)	`math.isclose(x, 3.14, rel_tol = 0.1)  # True`
math.exp(x)	Return e raised to the power x	`math.exp(x)  # 23.1407`
math.log(x[, base])	With one argument, return the natural logarithm of x (to base e). With two arguments, return the logarithm of x to the given base.	`math.log(x, 3)  # 1.0420`
math.pow(x, y)	Return x raised to the power y.	`math.pow(x, 2)  # 9.8696`
math.sqrt(x)	Return the square root of x.	`math.sqrt(x)  # 1.7725`
math.sin(x)/cos(x)/tan(x)	Return the sine/cosine/tangent of x	`math.sin(x/2)  # 1.0`

https://docs.python.org/3/library/math.html
Also, see Day 3 Slides

Standard Libraries/Packages/Modules

Python comes with extensive built-in libraries, packages and modules

- Mathematical Modules
  - **random** — Generate pseudo-random numbers

Name	Explanation	Example
random.randint(*a*, *b*)	Return a random integer N such that a <= N <= b.	`random.randint(10, 20)  # 12`
random.random()	Return the next random floating point number in the range [0.0, 1.0).	`random.random()  # 0.7379`
random.sample(population, k)	Return a k length list of unique elements chosen from the population sequence or set. (K should be smaller than the size of population)	`random.sample([10, 20, 30, 40, 50], k=4)` `# [20, 50, 40, 10]`
random.shuffle(x)	Shuffle the sequence x in place.	`x = [10, 20, 30, 40, 50]` `random.shuffle(x) # x becomes [20, 40, 50, 30, 10]`

https://docs.python.org/3/library/random.html

Contents

Modules, Packages and Libraries

Importing Libraries/Packages/Module

Standard Libraries/Packages/Modules

- Mathematical Modules
- **Data Persistence**
- File Formats
- Generic Operating System Services

Standard Libraries/Packages/Modules

Python comes with extensive built-in libraries, packages and modules

- Data Persistence
  - **pickle** — Python object serialization
  - copyreg — Register pickle support functions
  - shelve — Python object persistence
  - marshal — Internal Python object serialization
  - dbm — Interfaces to Unix “databases”
  - sqlite3 — DB-API 2.0 interface for SQLite databases

https://docs.python.org/3/library/

Object Serialization

Serialization: Process of converting an object to a stream of bytes to store it in the disk.

- Pickle is the native data serialization module and often used to store data, machine learning models, etc.

Syntax

- Serialization

Name	Explanation	Example

pickle.dumps(obj)	Return the pickled representation of the object obj as a bytes object (not a file).	`pickeld_data = pickle.dumps(input)`
		`print(pickeld_data)  # prints`
		`b'\x80\x04\x95\x0b\x00\x00\x00\x00\x00\x00\x00]\x94(K\x01K\x02K\x03e.'`

pickle.dump(obj, file)	Write the pickled representation of the object obj to the open file object, file.
	Note : file is not a file name string, it is a file object. You need to open it in a binary format (wb, rb).
		`input = [1, 2, 3]`
		`file_name = "file.pickle"`
		
		`with open(file_name, 'wb') as file:`
		`    pickle.dump(input, file)  # creates pickled file_name`

https://docs.python-guide.org/scenarios/serialization/
https://docs.python.org/3/library/pickle.html

Object Serialization

Serialization: Process of converting an object to a stream of bytes to store it in the disk.

- Pickle is the native data serialization module and often used to store data, machine learning models, etc.

Syntax

- De-serialization

Name	Explanation	Example

pickle.loads(data)	Return the reconstituted object from the pickled data.	`pickled_data = b'\x80\x04\x95\x0b\x00\x00\x00\x00\x00\x00\x00]\x94(K\x01K\x02K\x03e.'`
		`data = pickle.loads(pickeld_data)`
		`print(data)  # print [1, 2, 3]`

pickle.load(file)	Read the pickled object from the open file object, file and return the reconstituted object.
	Note : file is not a file name string, it is a file object. You need to open it in a binary format (wb, rb).
		`file_name = "file.pickle"`
		
		`with open(file_name, 'rb') as file:`
		`  data = pickle.load(file)`
		`  print(data)  # print [1, 2, 3]`

https://docs.python-guide.org/scenarios/serialization/
https://docs.python.org/3/library/pickle.html

Example 2

Create return_avg_air_quality(file_name, aqi_range, site_id, abs_tol) which loads a pickle file called file_name and returns 'The average aqi is XX, which is YY' (Ex. 'The average aqi is 13.7, which is Good') based on the aqi_range and abs_tol for the given site_id.

- You can assume that aqi_range increase by 50, and use 50 for default value of abs_tol.
- file_name: A name of a pickle file. This includes a Python array including rows with year, month, day, site_id, daily_pm10_concentration, daily_aqi_value, site_name, state, county, site_latitude, and site_longitude.
- Ex.

```
return_avg_air_quality(file_path, site_id, aqi_range,  abs_tol=50)
returns 'The average aqi is 15.9118, which is Good'
```

https://www.airnow.gov/aqi/aqi-basics

Contents

Modules, Packages and Libraries

Importing Libraries/Packages/Module

Standard Libraries/Packages/Modules

- Mathematical Modules
- Data Persistence
- **File Formats**
- Generic Operating System Services

Standard Libraries/Packages/Modules

Python comes with extensive built-in libraries, packages and modules

- File Formats
  - **csv** — CSV File Reading and Writing
  - configparser — Configuration file parser
  - netrc — netrc file processing
  - plistlib — Generate and parse Apple .plist files

https://docs.python.org/3/library/

CSV File Reading and Writing

CSV (Comma Separated Values) is a common data representation format.

- Each row has column values separated by commas (,)

The CSV module offers reading and writing data from a csv file.

Name	Explanation	Example

csv.writer(csvfile)	Return a writer object responsible for converting the user's data into delimited strings on the given file-like object. If csvfile is a file object, it should be opened with a parameter, newline=''. Once creating a csv writer object, you can write each row by csv_writer.writerow([col1, col2, ...]) or csv_writer.writerows(rows) **Note** : csvfile is not a file name string, it is a file object.	`input = [[1, 2], [3, 4]]`
		`file_name = "integer.csv"`
		`with open(file_name, "w") as file:`
		`  csv_writer = csv.writer(file)`
		`  for row in input:`
		`  csv_writer.writerow(row)`
		`  # the above is equivalnet to the following`
		`  csv_writer.writerows(input)`

csv.reader(csvfile)	Return a reader object which will iterate over lines in the given csvfile. The reader object supports iterator protocol. Note : csvfile is not a file name string, it is a file object.	`with open(file_name, "r") as file:`
		`  csv_reader = csv.reader(file)`
		`  print(list(csv_reader))  # prints [['1', '2'], ['3', '4']]`

https://docs.python.org/3/library/csv.html

Example 3

Create the following function

- convert_pickle_to_csv() which creates a csv file in output_path, by converting the pickle file in input_path.
- filter_csv_and_save() which loads the csv file in input_path and filter data on index, and writes its unique set as a pickle file on output_path.

```
input_path = '../Data/epa_air_quality.pickle'
output_path = 'epa_air_quality.csv'
convert_pickle_to_csv(input_path, output_path)

input_path = 'epa_air_quality.csv'
output_path = 'epa_stations.pickle'
index = 3
filter_csv_and_save(input_path, output_path, index)
```

Contents

Modules, Packages and Libraries

Importing Libraries/Packages/Module

Standard Libraries/Packages/Modules

- Mathematical Modules
- Data Persistence
- File Formats
- **Generic Operating System Services**

Standard Libraries/Packages/Modules

Python comes with extensive built-in libraries, packages and modules

- Generic Operating System Services
  - **os** — Miscellaneous operating system interfaces
  - io — Core tools for working with streams
  - time — Time access and conversions
  - argparse — Parser for command-line options, arguments and sub-commands
  - logging — Logging facility for Python
  - getpass — Portable password input
  - curses — Terminal handling for character-cell displays
  - platform — Access to underlying platform's identifying data
  - errno — Standard errno system symbols
  - And many more

https://docs.python.org/3/library/

Operating System Interfaces

os module

- Provides operating system dependent functionality.

Name	Explanation	Example
os.system(shell_command)	Execute shell_command	`os.system('ls -la')  # performs 'ls -la'`
os.makedirs(name)	Create a folder with the given name.	`os.makedirs('dir') # creates a directory called 'dir'`
os.rename(src, dst)	Rename the file or directory src to dst.	`os.rename('dir','dir_2')  # rename 'dir' to 'dir_2'`
os.remove(file_path)	Remove (delete) the file on the file_path	`os.remove('file.pickle')  # delete 'file.pickle'`
os.rmdir(folder_path)	Remove (delete) the folder on the folder_path	`os.removedirs('dir_2')  # delete 'dir_2'`
os.environ(var_name)	Return the value of the environment variable, var_name	`os.environ['PATH'] # Returns environment variable, PATH`

https://docs.python.org/3/library/os.html
https://docs.python.org/3/library/os.path.html

Operating System Interfaces

os module

- `os.path` - includes useful functions on path names.

Name	Explanation	Example
os.path.exists(path)	Return True if path refers to an existing path.	`os.path.exists('./Skeleton')` `# True`
os.path.isfile(path)	Return True if path is an existing regular file.	`os.path.isfile('ex03.ipynb')  # True` `os.path.isfile('Skeleton')  # False`
os.path.isdir(path)	Return True if path is an existing regular directory (folder).	`os.path.isdir('ex03.ipynb')  # False` `os.path.isdir('./Skeleton')  # True`

https://docs.python.org/3/library/os.html
https://docs.python.org/3/library/os.path.html

Example 4

Let's re-write our HW1 as .py file (or .ipynb) instead of .sh.

- Read comma-separated lines from file_name.
- In this file, the first field is a url for the git repo and the second field is the folder path.
- If the folder is not created yet, clone the repository to the given folder path.
- If the folder exists (you can assume that it is a git repository), update the repo using git pull.

Contents

Modules, Packages and Libraries

Importing Libraries/Packages/Module

Standard Libraries/Packages/Modules

- Mathematical Modules
- Data Persistence
- File Formats
- Generic Operating System Services

References

Python : https://docs.python.org/3/